

Ressources

- <https://developer.mozilla.org/fr/docs/Web/JavaScript>
- https://developer.mozilla.org/fr/docs/Web/API/Document_Object_Model
- <http://www.toutjavascript.com/reference/index.php> - référence JavaScript
- <http://www.w3schools.com/> - tutoriaux JavaScript, HTML, CSS, etc. – en anglais
- “Javascript: The Definitive Guide: Master the World's Most-used Programming Language”. David Flanagan, 2020, ISBN : 9781491952023

JavaScript

- Langage de script - coté client
- Permet de rendre dynamiques les pages Web
- Permet de répondre aux actions de l'utilisateur
- Langage de programmation orienté objet

Pré-requis

- Notions d'algorithmique
- Bonne connaissance des langages :
 - HTML
 - CSS

Pourquoi JavaScript ?

- Quelques exemples
 - Validation de formulaires
 - https://developer.mozilla.org/fr/docs/Learn/Forms/Form_validation
 - Réaliser des galeries d'images animées
 - https://www.w3schools.com/HOWTO/tryit.asp?filename=tryhow_js_slideshow_gallery
 - Réaliser un simulateur de crédit immobilier
 - <https://www.mortgagecalculator.org/free-tools/javascript-mortgage-calculator.php>
 - Réaliser des animations
 - <http://leocs.me/jquery-drawsvg/#demos>

Objectif du cours

- Connaître les éléments de base du langage JavaScript pour rendre les pages Web plus :
 - intéressantes
 - attrayantes
 - interactives
 - utilisables

Les outils nécessaires

- Un éditeur de code simple pour visualiser ou éditer les sources JavaScript
- Un navigateur pour JavaScript coté client

Rappel HTML / CSS

```
<li class="fruit">framboise</li>
```

```
.fruit{color:pink;}
```

Script

- Un script est un ensemble d'instructions
- Les instructions s'exécutent pas à pas
- Il est possible d'exécuter différentes portions de code, en réponse à une situation

Exemple HTML

```
<!DOCTYPE html>
<html lang="fr">
  <head>
    <title>Programmation Web FrontEnd</title>
    <meta charset="utf-8" />
    <link rel="stylesheet" href="style.css" />
  </head>
  <body>
    <h1>Cours 1</h1>
    <script src="ajout-contenu.js"></script>
    <p>Posez vos questions dans le forum <em><a
href="https://moodle.univ-
lyon2.fr/course/view.php?id=23733">Cours Moodle
JS</a></em></p>
  </body>
</html>
```

Exemple JavaScript

```
var jour = new Date();
var heureActuelle = jour.getHours();
var salut;

if (heureActuelle > 18) {
    salut = 'Bonsoir !';
} else if (heureActuelle > 12) {
    salut = 'Bon après-midi !';
} else if (heureActuelle > 0) {
    salut = 'Bonjour !';
} else {
    salut = 'Bienvenue!';
}
document.write('<h2>' + salut + '</h2>');
```

Contenu interactif avec JS

- Accès au contenu
- Modification du contenu
- Règles de programmation
- Réponse à des événements

Notions de base du langage JS

- Java Script contient :
 - Des mots clés
 - Fonctions natives : `escape()`, `eval()`, `isFinite()`, `isNaN()`, `parseFloat()`, `parseInt()`, `unescape()`, ...
 - Instructions : `break`, `if`, `else`, `for`, `in`, `return`, `switch`, `while`, `with`
 - Opérateurs : `function`, `new`, `this`, `var`, `+`, `-`, `*`, `/`, ...
 - Des événements : `onclick`, `onload`, etc.
 - Des objets
 - Propriétés
 - Méthodes

Notions de base du langage JS

- Commentaires :
 - `//` commentaire sur une ligne
 - `/*` commentaire
sur plusieurs
lignes `*/`

Boîtes de dialogue

- **alert()** : Pour afficher un message simple à l'utilisateur.
- `alert("Bienvenue sur notre site !");`
- **prompt()** : Pour demander une entrée à l'utilisateur > 1 ou 2 paramètres
`prompt("Question à l'utilisateur",
"Texte par défaut");`
- `let nom = prompt("Quel est votre nom ?");`
- `alert("Bonjour, " + nom + " !");`

Afficher des Informations en JS

- Sortie de base, permet d'écrire directement du contenu dans le document HTML :
 - `document.write()` ;
- Exemple :
- `<script>`
- `document.write("Bonjour le monde !") ;`
- `</script>`
- `console.log()` est une méthode utilisée pour afficher des informations dans la console du navigateur (utile pour le débogage et le développement)
- `console.log("Message à afficher") ;`

Générer des balises HTML

- Quel est le contenu affiché par cette page Web ?

```
<html>
```

```
  <head>
```

```
  </head>
```

```
  <body>
```

```
    <h1>Titre</h1>
```

```
    <script>
```

```
      document.write("<strong>Bonjour le monde !</strong> <br />");
```

```
    </script>
```

```
      <p> test</p>
```

```
    </body>
```

```
</html>
```

Exemple

- Quel est le résultat de cette ligne ?
- `document.write(prompt("Entrez votre couleur préférée :", "bleu"));`

Types de données

- Nombres (type **Number**) : 17, 21.5, -50, -0.78, etc.
 - Valeurs permet de stocker des nombres décimaux entre 2^{-1024} et 2^{1024} , mais ne permet de représenter des entiers de façon sûre qu'au sein de l'intervalle allant de $-(2^{53} - 1)$ à $2^{53} - 1$
 - **NaN** (pour Not A Number en anglais, qui signifie « qui n'est pas un nombre ») est utilisée lorsque le résultat d'une opération arithmétique ne peut pas être exprimée comme un nombre. Il s'agit également de la seule valeur JavaScript qui n'est pas égale à elle-même
- Chaînes de caractères (type **String**) : " Bonjour ", 'test'
- **Boolean** : **true** (vrai) ou **false** (faux)
- **Null** : mot clé spécial qui représente l'absence de valeur
- **Undefined** - représente une valeur non initialisée
- **BigInt** - utilisé pour représenter des entiers qui dépassent la limite de précision des nombres primitif (plus grands que $2^{53} - 1$)
 - `let bigNumber = BigInt(123456789012345678901234567890);`

Conversion de types de données

- Quel est le résultat de l'évaluation de ces expressions ?
- $2+5$
- $2+"5"$
- $2+3+"5"$
- $2+3+"5"+2*3$
- $2+3+"5"+2-3$
- **typeof** : Pour vérifier le type d'une variable :
- ```
console.log(typeof 42); // "number"
console.log(typeof "Salut"); // "string"
console.log(typeof true); // "boolean"
console.log(typeof undefined); // "undefined"
```

# Conversion de types de données

- Conversion de chaînes de caractères en nombres
- Les fonctions :
  - **parseInt("chaîne")** – transforme la chaîne de caractères en entier
  - **parseFloat("chaîne")** - transforme la chaîne de caractères en réel
  - **Number("chaîne")**
- Conversion de nombres en chaînes de caractères ?

# Déclaration des variables

- La commande **var** ou **let** (recommandé) permet de déclarer des variables
  - `let strExemple = " un texte ";`
  - `document.write(strExemple);`
- Noms des variables : doivent commencer par une lettre ou par "\_". Les caractères suivants peuvent inclure des nombres
- **const** – pour déclarer des constantes

# Affectation de valeurs

- `=` attribue la valeur de l'opérande droit à l'opérande gauche
- `+=` ajoute les opérandes gauche et droit et affecte le résultat à l'opérande gauche
- `-=` (soustraction), `*=` (multiplication), `/=` (division)
- `%=` divise l'opérande gauche par l'opérande droit et affecte le reste à l'opérande gauche

# Opérateurs

- Arithmétiques binaires : +, -, \*, /, %
- Arithmétiques unaires : ++ (incrément), -- (décrément), -(négation unaire)
- Exemple :
  - ++x incrémente x d'une unité et renvoie le résultat
  - x++ renvoie la valeur de x et incrémente ensuite

# Exemple

- Quelles sont les valeurs des variables suivantes après exécution du script suivant ?
- `x = 5;`
- `y = ++x ;`
- `z = x++;`
- `document.write(x+"<br />");`
- `document.write(y+"<br />");`
- `document.write(z+"<br />");`

# Opérateurs logiques

- `&&` - And (et) logique renvoie la valeur true lorsque les 2 opérandes ont pour valeur true et false dans les autres cas
- `||` - Or (ou) logique renvoie true lorsque l'un des opérandes a pour valeur true et false lorsque les deux opérandes ont pour valeur false
- `!` – Not logique – renvoie la valeur true si l'opérande a pour valeur false et false si l'opérande a pour valeur true (opérateur unaire: il précède l'opérande)

# Opérateurs de comparaison

- `==` (égalité) et `===` (égalité stricte : valeur et type)
- `!=` différence
- `>`, `<`, `>=`, `<=`
- Exemples :
  - `1==1 //true`
  - `3<1 //false`
  - `5 === "5" // false` (types différents : nombre et chaîne)
  - `"5" < 10 // true` (la chaîne "5" est convertie en nombre 5)
  - `true == 1 // true` (true est converti en 1)

# Chaînes de caractères

Spécifier une chaîne de caractères :

- l'entourer de guillemets simples (le caractère ') ou doubles (le caractère ")
- vous pouvez utiliser des guillemets dans une chaîne, tant qu'ils ne correspondent pas aux caractères qui entourent la chaîne

•Exemple :

- "C'est une belle journée"
- 'Le pseudonyme de Pablo Ruiz est "Picasso" '

# Chaînes de caractères

- Pour spécifier des guillemets vous pouvez aussi les précéder d'un antislash (\)
- Pour spécifier un antislash littéral avant un guillemet simple ou à la fin d'une chaîne de caractères, écrivez-le deux fois (\\)
- \" → "
- \' → '
- \\ → \
- \n → nouvelle ligne
- \r → retour chariot
- \t → tabulation

# Chaînes de caractères

Parmi ces valeurs, quelles sont des chaînes de caractères correctement écrites ?

1. `strTexte = 'Je m'appelle' ;`
2. `strTexte = "Je m'appelle" ;`
3. `strTexte = 'Je m\'appelle' ;`
4. `strTexte = 'Mes "super" amis' ;`
5. `strTexte = "Mes "super" amis" ;`
6. `strTexte = "Mes \"super\" amis" ;`
7. `strTexte = "Mes \"super\" amis" ;`
8. `alert("strTexte =" + strTexte) ;`

# Chaînes de caractères

- La propriété **length**
  - Retourne la longueur d'une chaîne de caractères
- Exemple :
  - `strCode="69003";`
  - `document.write(strCode.length);`

# Chaînes de caractères

- Modifier la casse
- **toLowerCase()**
  - Retourne la chaîne avec tous les caractères en minuscules
  - Exemple : `str2=str1.toLowerCase()` ;
- **toUpperCase()**
  - Retourne la chaîne avec tous les caractères en majuscules
  - Exemple : `str2=str1.toUpperCase()` ;

# Chaînes de caractères

- Recherche de sous-chaînes
- **Integer chaîne.indexOf(String souschaîne[, Integer debut])**
  - Retourne la position d'une sous-chaîne *souschaîne* dans une chaîne, à partir de la position *debut*. Si la sous-chaîne n'est pas trouvée, `indexOf()` retourne -1. Sensible à la casse.
  - Si *debut* absent, par défaut il a la valeur 0.
- **Integer chaîne.lastIndexOf(String souschaîne)**
  - Retourne l'indice de la dernière occurrence de souschaîne.
  - Si souschaîne n'est pas trouvée, `lastIndexOf()` retourne -1.
- **String chaîne.slice(Integer debut [, Integer fin])** ou
- **String chaîne.substring(Integer debut [, Integer fin])**
  - Extrait une sous-chaîne d'une chaîne en partant de l'indice debut et jusqu'à l'indice fin.
  - Si fin n'est pas précisé, la chaîne est extraite depuis le début jusqu'à sa fin.
- **String chaîne.substr(Integer debut, Integer longueur)**
  - Presque identique à `substring()`.
  - La seule différence tient dans le second paramètre qui indique le nombre de caractères à extraire.

# Tableaux

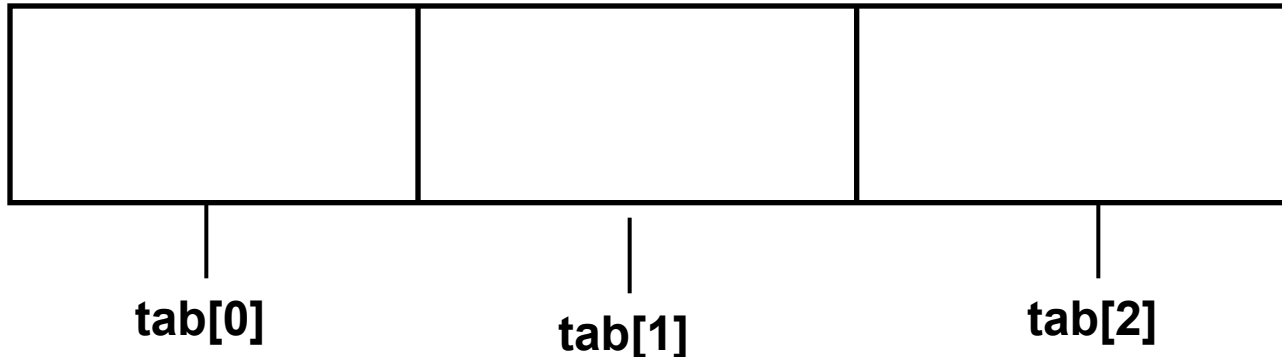
- Un tableau est une variable qui permet de stocker plusieurs données différentes, rangées en mémoire les unes à la suite des autres.
- Exemples :
  - `var totaux_mensuels = new Array(12);`
  - `totaux_mensuels[2] = 2000.30;`
  - `totaux_mensuels[3]='pas de ventes';`
  - `var jours = new Array("Lun", "Mar", "Mer", "Jeu", "Ven", "Sam", "Dim");`

# Tableaux

- **new Array** effectue 2 tâches différentes :
  - Calcule le nombre d'éléments présents dans le tableau et le place dans un espace réservé appelé **length**.
  - Libère suffisamment d'espace de mémoire pour tous les éléments du tableau et associe cet espace à la variable tableau ainsi définie.

# Tableaux unidimensionnels

- Attention : l'indexation des tableaux commence à 0 !
- Les éléments du tableau n'ont pas forcément le même type
- Exemple : **var tab = new Array(3) ;**



# Tableaux

- Taille d'un tableau :
  - *NomTableau*.length
  - Exemple : **tab.length**
- Accéder à un élément du tableau :
  - *NomTableau*[*index*]
  - Exemple : **tab[0]** , **tab[2]** , ...

# Tableaux à plusieurs dimensions

- Dans un élément de tableau on peut stocker un autre tableau
- Exemple :

- `var matrice = new Array(2);`
- `matrice[0] = new Array(2);`
- `matrice[1] = new Array(2);`
- `matrice[0][0] = 1;`
- `matrice[0][1] = 0;`
- `matrice[1][0] = 0;`
- `matrice[1][1] = 1;`

|           |           |
|-----------|-----------|
| Tab[0][0] | Tab[0][1] |
| Tab[1][0] | Tab[1][1] |

# La structure if ... else

- `if (condition vraie) {  
    instruction;  
}`
- Si la condition est vérifiée, les instructions s'exécutent. Si elle ne l'est pas, les instructions ne s'exécutent pas et le programme passe à la commande suivant l'accolade de fermeture
- `if (condition vraie) {  
    instructions1;  
}  
else {  
    instructions2;  
}`
- Si la condition est vérifiée (true), le bloc d'instructions1 s'exécute. Si elle ne l'est pas (false), le bloc d'instructions2 s'exécute.
- Dans le cas où il n'y a qu'une instruction, les accolades sont facultatives.

# Boucles

- `while ( expression ) {  
    instruction(s) ;  
};`
- `for (expression1; expression2;  
    expression3) {  
    instruction(s) ;  
};`

# while

- Permet de répéter une instruction ou un bloc d'instructions autant de fois qu'on désire, en testant à chaque fois une condition.
- ```
while ( expression ) {  
    instruction(s) ;  
};
```
- Comment ça marche ?
 1. L'*expression* est évaluée
 2. Si elle est non nulle, l'*instruction* s'exécute et retour à 1. Si elle est nulle, l'exécution continue **après** *instruction*.

while - Exemple

- Afficher les entiers de 1 à 10

```
<html>
  <head>
  </head>
  <body>
    <script>
      var nPos = 1;
      while (nPos <= 10)
      {
        document.write("nPos = " + nPos + "<br />");
        nPos = nPos + 1;
      }
    </script>
  </body>
</html>
```

for

- **for** (*expression1*; *expression2*; *expression3*)
 instruction;

équivalent à :

- *expression1*;
while (*expression2*) {
 instruction;
 expression3;
}

for - Exemple

- Afficher les entiers de 1 à 10.

```
<html>
  <head>
  </head>
  <body>
    <script>
      for(nPos = 1; nPos <=10; nPos++)
        document.write("nPos="+nPos+"<br />");
    </script>
  </body>
</html>
```

Exemple - for

```
var fruit = new Array(3);
```

```
fruit[0] = 'pomme';  
fruit[1] = 'poire';  
fruit[2] = 'orange';
```

```
for (i=0; i<=fruit.length; i++)  
    document.write(fruit[i] + ' ');
```

Mots clés JavaScript

- abstract boolean break byte
case catch char class const continue
debugger default delete do double
else enum export extends
false final finally float for function goto
if implements import in instanceof int interface
let long native new null
package private protected public return
short static super switch synchronized
this throw throws transient true try typeof
var void volatile while with yield